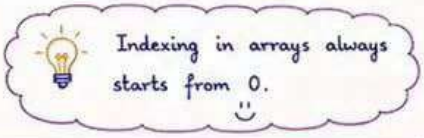
DSA NOTES - 1. ARRAYS

1. What is an Array?

An array is a collection of elements of same data type stored in contiguous memory locations.

Elements are accessed using index.



2. Example

arr = [10, 20, 30, 40, 50]

10	20	30	40	50
0	1	2	3	4

index : 0 1 2 3 4

3. Characteristics / Properties

- Fixed size (size decided at creation).
- Elements are of same data type.
- Stored in contiguous memory locations.
- Random access is possible in O(1) time.
- Duplicate elements are allowed.

4. Why Arrays?

- ☑ Random access in O(1).
- ☑ Easy to traverse and implement.
- ☑ Cache friendly (better performance).
- ☑ Used as a base for many data structures (Stack, Queue, Heap, etc.)

5. Declaration (Examples)

In C / C++ :

```
int arr[5];
int arr[5] = {1, 2, 3, 4, 5};
int arr[] = {1, 2, 3, 4, 5}; // size auto
int arr[5] = {0};            // all elements 0
vector<int> arr(5);          // in C++ STL
```

In Java :

```
int[] arr = new int[5];
int[] arr = {1, 2, 3, 4, 5};
```

6. Accessing Elements

Formula :

Address of arr[i] = Base Address + (i * Size of data type)

Example :

```
int arr[5] = {10, 20, 30, 40, 50};
```

Element at index 2 → arr[2] = 30

First element → arr[0] = 10

Last element → arr[n-1] = arr[4] = 50

7. Basic Operations on Arrays

(1) Traversal

```
for (int i = 0; i<n; i++)
    cout << arr[i] << " ";

// Time Complexity: O(n)
```

(2) Insertion

```
// Insert x at index pos
for (int i = n; i> pos; i--)
    arr[i] = arr[i-1];
arr[pos] = x;
n++;
// Time Complexity: O(n)
```

(3) Deletion

```
// Delete element at index pos
for (int i = pos; i<n-1; i++)
    arr[i] = arr[i+1];
n--;

// Time Complexity: O(n)
```

(4) Searching (Linear)

```
for (int i = 0; i<n; i++) {
    if (arr[i] == key)
        return i;
}
return -1;

// Time Complexity: O(n)
```

(5) Update

```
arr[pos] = newValue;

// Time Complexity: O(1)
```

8. Types of Arrays

- 1D Array → Linear list of elements.
- 2D Array → Matrix (rows x columns).
- Multidimensional Array → 3D, 4D, ...
- Jagged Array → Array of arrays with different sizes.

1D Array

arr = [1, 2, 3, 4, 5]

index 0 1 2 3 4

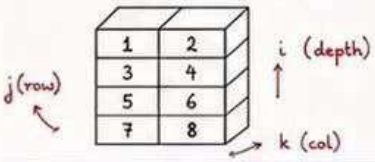
2D Array

arr = [[1, 2, 3], [4, 5, 6]]

	col 0	col 1	col 2
row 0	1	2	3
row 1	4	5	6

3D Array (2x2x2)

arr = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]



9. Common Problems on Arrays

- Find Maximum / Minimum element
- Find Second Largest / Smallest
- Check if Array is Sorted
- Remove Duplicates (in sorted array)
- Left Rotate / Right Rotate
- Move All Zeros to End
- Find Union / Intersection of Two Arrays
- Kadane's Algorithm (Maximum Subarray Sum)

10. Time & Space Complexity of Basic Operations

Operation	Time Complexity	Space Complexity
Access	O(1)	O(1)
Search	O(n)	O(1)
Insertion	O(n)	O(1)
Deletion	O(n)	O(1)
Traversal	O(n)	O(1)

11. Advantages & Disadvantages

Advantages

- ☑ Random access in constant time.
- ☑ Easy to implement.
- ☑ Cache friendly.
- ☑ Used in many Data Structures.

Disadvantages

- ☒ Fixed size (in static arrays).
- ☒ Insertion / Deletion costly.
- ☒ Wastage of space (if under-utilized).

Quick Tips

- ★ Use arrays when size is known and fixed.
- ★ Prefer arrays for simple data and better performance.

Think Smart, Code Better, Optimize Always!

Practice More

- ★ Solve problems on traversal, searching, rotation, subarrays, etc.
- ★ Build strong logic using arrays.

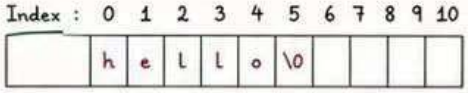
DSA NOTES - 2. STRINGS

1. What is String?

- A string is a sequence of characters terminated by a null character '\0'.
- In C/C++, strings are stored as array of characters.

Strings are immutable in many languages (like Java, Python). But in C/C++, they are mutable.

2. Representation



String : "hello"

Length : 5

(Length does not include '\0')

3. Declaration (Examples)

In C :

```
char str[20];
char str[ ] = "hello";
char str[20] = "hello";
```

In C++ :

```
string s;              // declaration
string s = "hello"; // initialization
```

4. Input / Output

In C :

```
char str[20];
printf("Enter string: ");
scanf("%s", str);   // input
printf("%s", str);  // output
```

In C++ :

```
string s;
cout << "Enter string: ";
getline(cin, s);   // input (can take spaces)
cout << s;         // output
```

5. Common String Functions (C/C++ STL)

• length(), size()	// length of string	O(1)
• empty()	// check if string is empty	O(1)
• clear()	// clear string	O(1)
• push_back(ch)	// add character at end	O(1)
• pop_back()	// remove last character	O(1)
• substr(pos, len)	// substring from pos, length len	O(n)
• find(sub)	// first index of substring	O(n)
• rfind(sub)	// last index of substring	O(n)
• find(ch, pos)	// find character from pos	O(n)
• replace(pos, len, str)	// replace substring	O(n)
• compare(s)	// compare strings	O(n)

6. Common Operations

• Traversal	// Access each character	O(n)
• Insertion	// Insert at any position	O(n)
• Deletion	// Delete at any position	O(n)
• Search	// Search character / substring	O(n)
• Concatenation	// Join two strings	O(n)
• Comparison	// Compare two strings	O(n)
• Reversal	// Reverse the string	O(n)

7. Important Algorithms on Strings

• Reverse a String	O(n)
• Check Palindrome	O(n)
• Check Anagram	O(n log n)
• Count Vowels and Consonants	O(n)
• Remove Duplicates	O(n)
• Longest Common Prefix (LCP)	O(n)
• Pattern Searching (KMP, Rabin-Karp)	O(n)

8. Examples

(1) Length of String

str = "hello"
length = 5

(2) Reverse String

str = "hello"
rev = "olleh"

(3) Check Palindrome

str = "madam"
Reverse = "madam"
Palindrome = Yes

(4) Check Anagram

s1 = "listen"
s2 = "silent"
Anagram = Yes

(5) Count Vowels

str = "abhishek"
Vowels (a,e,i,o,u) = 4

(6) Remove Duplicates

str = "programming"
Result = "progmin"

9. String Traversal

In C :

```
char str[100];
for(int i = 0; str[i] != '\0'; i++){
    cout << str[i] << " ";
}
```

In C++ :

```
string s = "hello";
for(int i = 0; i < s.length(); i++){
    cout << s[i] << " ";
}
```

10. Comparison of Strings

In C (strcmp) :

```
int strcmp(const char* s1, const char* s2);
```

- Returns 0 → s1 == s2
- Returns < 0 → s1 < s2
- Returns > 0 → s1 > s2

In C++ :

```
s1 == s2    // true if equal
s1 != s2    // true if not equal
s1 < s2     // lexicographical comparison
```

11. Tips

- ☑ Strings are widely used in interviews.
- ☑ Practice standard problems.
- ☑ Know library functions.
- ☑ Be careful with index (0-based).
- ☑ For taking input with spaces, use getline() in C++.

Think Smart;
Code Better,
Optimize Always!

12. Time & Space Complexity of Common Operations

Operation	Time Complexity	Space Complexity	Note
Access (by index)	O(1)	O(1)	Direct access
Traversal	O(n)	O(1)	Visit each character
Search (char / substr)	O(n)	O(1)	Linear search
Insertion / Deletion	O(n)	O(1)	May shift characters
Concatenation	O(n)	O(n)	New string creation
Reversal	O(n)	O(n)	New / in-place reverse
Comparison	O(n)	O(1)	Lexicographical

DSA NOTES - 3. SLIDING WINDOW

1. What is Sliding Window?

Sliding Window is a technique used to solve problems on arrays / strings by maintaining a window (subarray / substring) and sliding it to explore the solution efficiently.

It helps to reduce time complexity by avoiding unnecessary recalculations.

2. Why Use Sliding Window?

- Reduces Time Complexity
- Avoids recomputation
- Very useful for range / subarray / substring problems
- Optimizes brute force solutions
- Works great on sorted / unsorted data

Types of Window

1. Fixed Size Window (size is constant)
2. Variable Size Window (size changes)

3. How Does It Work?

Step 1: Define two pointers → left, right
Step 2: Expand the window by moving right
Step 3: Update the result as per condition
Step 4: Shrink the window by moving left
Step 5: Repeat until right reaches the end



4. Template (Generic Code)

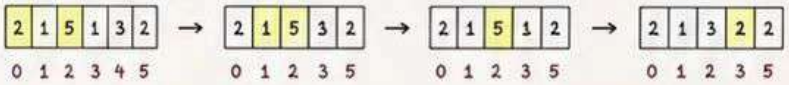
```
left = 0
result = 0
for right in range(n):
    add arr[right] / include current element
    while window is not valid:
        remove arr[left] / exclude left element
        left += 1
    update result
```

right → expands the window
left → shrinks the window
Window is valid → based on problem condition

5. Fixed Size Window (Size is Constant)

Example: Maximum Sum Subarray of Size K

arr = [2, 1, 5, 1, 3, 2], k = 3



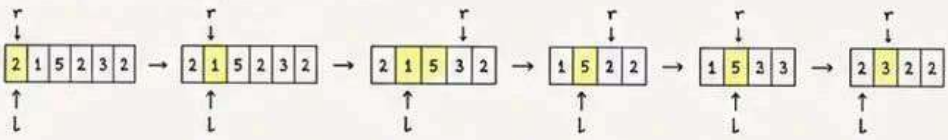
sum = 8, max = 8
sum = 7, max = 8
sum = 9, max = 9
sum = 6, max = 9

Maximum Sum = 9

6. Variable Size Window (Size Changes Dynamically)

Example: Smallest Subarray with Sum >= S

arr = [2, 1, 5, 2, 3, 2], S = 7



sum = 2 (< 7)
sum = 3 (< 7)
sum = 8 (>= 7), len = 3
sum = 7 (>= 7), len = 3
sum = 10 (>= 7), len = 3
sum = 7 (>= 7), len = 3

Smallest Length = 3 (Subarray: [5,2] or [2,3,2])

7. Common Problems Solved Using Sliding Window

★ Maximum Sum / Average of Subarray of Size K
★ Longest Substring Without Repeating Characters
★ Smallest Substring with Sum >= K
★ Fruits into Baskets (At most 2 distinct)
★ Number of Occurrences of Anagrams
★ Maximum Consecutive Ones (after replacement)
★ Longest Repeating Character Replacement

8. Comparison

Technique	Best For	Time Complexity	Space Complexity
Brute Force	Small input size, simple problems	$O(n^2)$ or more	$O(1)$
Sliding Window	Range / subarray / substring problems	$O(n)$	$O(1)$
Prefix Sum	Sum queries in subarrays	$O(n)$ preprocessing + $O(1)$ query	$O(n)$

9. Tips

- ☑ Understand the condition to expand and shrink the window.
- ☑ Always think: when to expand (right++) and when to shrink (left++).
- ☑ Initialize window and result properly.
- ☑ Dry run on small examples.
- ☑ Choose Fixed or Variable window as per the problem.
- ☑ Sliding Window is all about smart movement, not brute force!

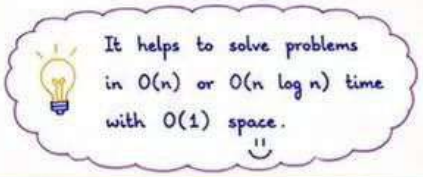
10. Complexity

- Time Complexity : $O(n)$ (Each element is visited at most twice)
- Space Complexity : $O(1)$ (Most problems)

DSA NOTES - 4. TWO POINTERS

1. What is Two Pointers?

Two Pointers is a technique that uses two indices (pointers) to traverse a data structure (usually arrays or strings) to solve problems efficiently.



2. Why Use Two Pointers?

- Reduces time complexity.
- Eliminates the need for extra space.
- Efficient for sorted arrays / strings.
- Useful for in-place problems.
- Works great for searching pairs / triplets, subarrays, partitions, etc.

3. Types of Two Pointers

1. Opposite Direction (Start & End)
   One pointer at start, other at end.
2. Same Direction (Sliding Window)
   Both pointers move in the same direction.
3. Slow and Fast Pointers
   One pointer moves slow (1 step), other moves fast (2 steps).

4. How It Works? (Opposite Direction)

1. Initialize one pointer at the beginning (left = 0) and another at the end (right = n-1).
2. Based on the condition, move the pointers.
3. Repeat until left < right.

Example Array

Index:	0	1	2	3	4	5	6
Arr :	2	4	6	8	10	12	14
	↑						↑
	left (start)						right (end)

5. Generic Template (Opposite Direction)

```
left = 0
right = n - 1
while left < right:
    if condition:
        # move pointers accordingly
    else:
        # move pointers accordingly
return result
```

6. Common Problems Solved Using Two Pointers

★ Pair with given sum in sorted array
★ Remove duplicates from sorted array
★ Palindrome check
★ Merge two sorted arrays
★ Container With Most Water
★ 3Sum / 4Sum (using sorting + two pointers)
★ Trapping Rain Water
★ Partition Array / Dutch National Flag
★ Linked List Cycle (Slow & Fast Pointers)

7. Examples

(1) Pair with given sum in sorted array

Arr = [2, 4, 6, 8, 10, 12, 14], target = 16
left = 0 (2), right = 6 (14) → sum = 16 ☑

Pair found: (2, 14)

(2) Remove duplicates from sorted array

Arr = [1, 1, 2, 2, 2, 3, 3, 4, 5, 5]
Use slow & fast pointers
Slow points to place of unique element.
Fast explores the array.

Result: [1, 2, 3, 4, 5]

(3) Palindrome Check

Str = "racecar"
left = 0, right = n-1
Compare str[left] and str[right]
If all match → Palindrome

Result: True

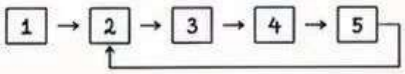
(4) Container With Most Water

Arr = [1, 8, 6, 2, 5, 4, 3, 7]
Move pointers inward and calculate max area.

Result: 49

8. Slow and Fast Pointers (Cycle Detection)

Floyd's Tortoise and Hare Algorithm



slow moves 1 step
fast moves 2 steps
If they meet → cycle exists.

Time Complexity: O(n)
Space Complexity: O(1)

9. Code Example (Pair with given sum)

```
# Python Code
def pair_with_sum(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return (arr[left], arr[right])
        elif s < target:
            left += 1
        else:
            right -= 1
    return None
```

10. Advantages

☑ Efficient (O(n) time)
☑ Space Optimized (O(1) space)
☑ Easy to implement
☑ Works well on sorted data

11. Tips

★ Make sure the array is sorted (if required).
★ Be careful with pointer movement.
★ Handle edge cases (empty array, single element, duplicates).
★ Practice more problems to get comfortable.

12. When to Use Two Pointers?

☑ When the array is sorted.
☑ When you need pairs/triplets with given condition.
☑ When you need to reduce space complexity.
☑ When you are working with linked lists (slow-fast).

★ Two Pointers = Simplicity + Efficiency
Master this technique to solve many interview level problems easily!

Remember:

Brute Force takes you to the solution,
Two Pointers takes you there faster!



DSA NOTES - 5. GREEDY APPROACH

1. What is Greedy Approach ?

- Greedy is an algorithmic paradigm that builds a solution step by step.
- At each step, it chooses the locally optimal choice with the hope of finding a global optimum.



2. Characteristics

- Makes locally optimal choice at each step.
- Choice is irrevocable (can't go back).
- Reduces time complexity.
- Optimal for problems with greedy-choice property and optimal substructure.
- Usually easier and faster than other approaches (DP, Backtracking).

3. When to Use Greedy ?

- When the problem has optimal substructure.
- When the problem has greedy-choice property.
- When taking the best immediate decision leads to global optimum.

Greedy-Choice Property : A globally optimal solution can be reached by selecting the best choice at each step.

Optimal Substructure : An optimal solution to the problem contains optimal solutions to its subproblems.

4. Generic Greedy Algorithm

Step 1 : Sort / arrange the input (if needed)
Step 2 : Initialize the result
Step 3 : While there are candidates make the best choice
Step 4 : Add the choice to result
Step 5 : Update the candidates
Step 6 : Repeat until no candidates left
Step 7 : Return the result

Template (Pseudo Code)

```
sort(input)
result = []
while candidates exist:
    choose best candidate
    result.add(candidate)
    update candidates
return result
```

5. Common Problems Solved Using Greedy

- ★ Activity Selection
- ★ Fractional Knapsack
- ★ Huffman Coding
- ★ Dijkstra's Algorithm
- ★ Minimum Spanning Tree (Kruskal / Prim)
- ★ Job Sequencing with Deadlines
- ★ Candy Distribution
- ★ Coin Change (Min Coins)
- ★ Interval Scheduling
- ★ Gas Station
- ★ Jump Game
- ★ Indian Coins (canonical systems)

6. Examples

(1) Activity Selection Problem

Given activities with start & finish times, select maximum activities that don't overlap.

Greedy Choice : Choose activity with earliest finish time.

Example :

Activity	Start	Finish
A	1	3
B	3	4
C	0	6
D	5	7
E	8	9
F	5	9

Result : A, B, D, E

(2) Fractional Knapsack

Given items with weight & value, put items in knapsack (capacity W) to maximize value. (Can break items)

Greedy Choice : Choose item with highest value/weight ratio.

Example : W = 50

Item	Weight	Value	Ratio
1	10	60	6.0
2	20	100	5.0
3	30	120	4.0

Pick : Item 1 (10), Item 2 (20), 2/3 of Item 3 (20)

Max Value = 240

(3) Huffman Coding

Given characters with frequencies, construct a prefix code with minimum cost.

Greedy Choice : Always combine two least frequent nodes.

Example : freq = {a:5, b:9, c:12, d:13, e:16, f:45}

Build Huffman Tree by combining smallest two repeatedly.

Result : Minimum cost encoding.

(4) Coin Change (Min Coins)

Given coin denominations, find minimum coins to make given amount.

Greedy Choice : Choose largest coin <= remaining amount.

Example : Coins = {1, 2, 5, 10}
Amount = 27

Pick → 10, 10, 5, 1, 1

(Total = 5 coins)

Result : 5 coins

(5) Job Sequencing with Deadlines

Each job has deadline & profit. Schedule jobs to maximize profit.

Greedy Choice : Choose job with highest profit first.

Example :

Job	Deadline	Profit
J1	2	100
J2	1	19
J3	2	27
J4	1	25
J5	3	15

Result : J1, J3, J5 (Max Profit = 142)

7. Advantages

- ☑ Simple and easy to understand.
- ☑ Usually faster (better time complexity).
- ☑ Works well for real-world problems.
- ☑ Provides optimal solution for problems with greedy-choice property.

8. Disadvantages

- ☒ Doesn't always give optimal solution.
- ☒ Greedy-choice property is not easy to prove.
- ☒ Not suitable for problems without optimal substructure.

9. Greedy vs Dynamic Programming

Aspect	Greedy	Dynamic Programming
Approach	Make local best choice	Solve all subproblems
Choices	Irrevocable	Reversible
Optimality	Not always optimal	Always optimal
Time Complexity	Usually lower	Usually higher
Space Complexity	Lower	Higher
Best For	When greedy works	General problems

10. Tips

- ★ Always try to prove greedy-choice property.
- ★ Sort the data if required.
- ★ Handle edge cases.
- ★ Practice more problems to identify greedy patterns.

11. Common Pitfalls

- Taking greedy approach on problems without greedy-choice property.
- Not checking if local choice leads to global optimum.
- Ignoring edge cases (empty input, single element, duplicates).

★ Greedy is not about finding perfect solution, it's about making the best possible choice right now !

Remember :

Greedy = Simplicity + Efficiency

Use it wisely, prove it carefully, and optimize your solution !



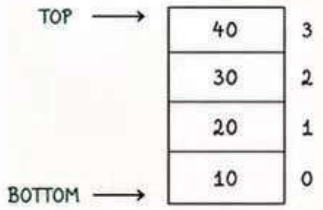
DSA NOTES - 6. STACK

1. What is Stack ?

- Stack is a linear data structure that follows LIFO (Last In First Out) order.
- Insertion and Deletion operations are allowed only at one end called TOP.
- It can be implemented using array or linked list.

Think of Stack like a plate stack in canteen, the last plate you put is the first one you take out.

2. Representation



Top element = 40
Bottom element = 10

3. Basic Operations

- push(x) → Add element x on top of stack.
- pop() → Remove and return top element.
- peek() / top() → Return top element without removing.
- isEmpty() → Check if stack is empty.
- isFull() → Check if stack is full (for array implementation).
- size() → Return number of elements in stack.

4. Implementation

A. Using Array

```
# Initialize
stack = []    # or stack = [None] * N
top = -1
N = size of array

# Push
if top == N-1:
    print("Stack Overflow")
else:
    top += 1
    stack[top] = x

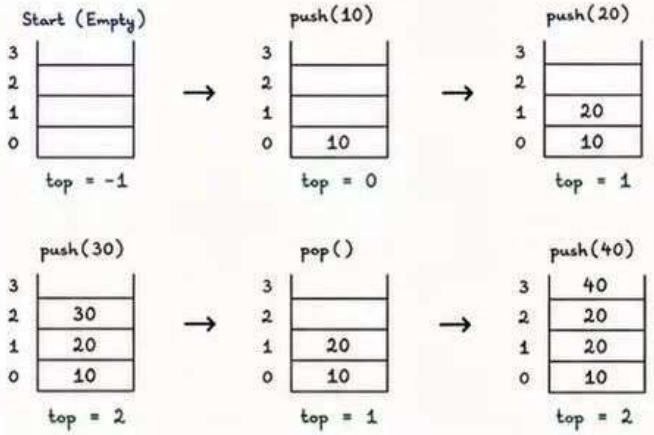
# Pop
if top == -1:
    print("Stack Underflow")
else:
    x = stack[top]
    top -= 1
    return x
```

B. Using Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None
    def push(self, x):
        node = Node(x)
        node.next = self.top
        self.top = node
    def pop(self):
        if not self.top:
            return None
        x = self.top.data
        self.top = self.top.next
        return x
```

5. Example

Perform push(10), push(20), push(30), pop(), push(40)



6. Applications of Stack

- ★ Function call management (Call Stack)
- ★ Expression evaluation (Infix, Prefix, Postfix)
- ★ Parentheses matching
- ★ Backtracking (DFS, Maze, N-Queens)
- ★ Undo operation in editors
- ★ Browser history (back button)

7. Types of Stack

A. Normal Stack
- Standard stack operations.

B. Min Stack
- Get minimum element in O(1) time.

C. Stack using two queues
- Implement stack using two queues.

D. Queue using two stacks
- Implement queue using two stacks.

8. Important Problems

- ★ Valid Parentheses
- ★ Next Greater Element
- ★ Largest Rectangle in Histogram
- ★ Evaluate Reverse Polish Notation
- ★ Min Stack
- ★ Basic Calculator

9. Complexities

Operation	Array Implementation	Linked List Implementation
push()	O(1)	O(1)
pop()	O(1)	O(1)
peek()	O(1)	O(1)
isEmpty()	O(1)	O(1)
isFull()	O(1)	O(n) / Not Required
size()	O(1)	O(1)

Master Stack,
Master Problem Solving!
Keep Practicing

10. Code (Stack using Python List)

```
stack = []

# Push
stack.append(x)

# Pop
if stack:
    stack.pop()
else:
    print("Stack Underflow")

# Peek
if stack:
    print(stack[-1])
else:
    print("Stack is Empty")

# isEmpty
len(stack) == 0
```

11. Quick Tips

- ☑ Always check for Underflow before pop.
- ☑ In array implementation, check for Overflow before push.
- ☑ Stack follows LIFO principle.
- ☑ Top is the most important pointer in stack.
- ☑ Use stack to reverse data.

"Code is like STACK,
Build it strong, use it smart!"

DSA NOTES - 8. SET / HASH SET

1. What is Set ?

- Set is a collection of unique elements.
- No duplicates are allowed.
- Elements in Set are unordered.
- Implemented using Hash Table.



Think of Set as a box that keeps only unique elements, order doesn't matter here !

2. Characteristics of Set

- ★ Stores only unique elements.
- ★ No insertion order (unless using LinkedHashSet).
- ★ Allows null (only one null element).
- ★ Basic operations in average O(1) time.
- ★ Uses Hashing for fast operations.

3. Common Set Operations

Operation	Description
add(x)	Insert x in the set.
remove(x)	Remove x from the set.
contains(x)	Check if x exists in the set.
size()	Return number of elements.
isEmpty()	Check if set is empty.
clear()	Remove all elements.

4. Set vs List/Array

Feature	Array / List	Set
Duplicates	Allowed	Not Allowed
Order	Maintains Order	Doesn't Maintain Order
Access	By Index (O(1))	No Index Access
Search	O(n)	O(1) avg (Hashing)
Use Case	When order matters	When unique elements are needed

5. Implementation in Java (HashSet)

```java
import java.util.*;
HashSet<Integer> set = new HashSet<>();

// Add
set.add(10);
set.add(20);
set.add(10); // duplicate → ignored
set.add(null); // allowed

// Operations
set.remove(20);
boolean exists = set.contains(10);
int s = set.size();
boolean empty = set.isEmpty();
set.clear();
```

6. Implementation in C++ (unordered_set)

```cpp
#include <bits/stdc++.h>
using namespace std;
unordered_set<int> st;

// Add
st.insert(10);
st.insert(20);
st.insert(10); // duplicate → ignored

// Operations
st.erase(20);
bool exists = st.find(10) != st.end();
int s = st.size();
bool empty = st.empty();
st.clear();
```

7. Example

Input Array : [4, 5, 4, 2, 5, 1, 2, 3]

Store in Set → {1, 2, 3, 4, 5} (Only unique elements)

Set helps in removing duplicates easily !

8. Common Applications of Set

- ★ Removing duplicates from array / string
- ★ Checking if an element exists or not
- ★ Finding union & intersection of two collections
- ★ Solving problems involving unique data
- ★ Counting distinct elements

9. Time Complexity (Average Case)

Operation	HashSet (Java)	unordered_set (C++)
Insert	O(1)	O(1)
Delete	O(1)	O(1)
Search	O(1)	O(1)
Traverse	O(n)	O(n)

(Worst Case: O(n) due to Hash Collision)

10. HashSet vs LinkedHashSet vs TreeSet (Java)

Feature	HashSet	LinkedHashSet	TreeSet
Order	No Order	Insertion Order	Sorted Order
Underlying DS	Hash Table	Hash Table + DLL	Red-Black Tree
Duplicates	Not Allowed	Not Allowed	Not Allowed
Null Element	One Null Allowed	One Null Allowed	No Restriction
Time Complexity	O(1) avg	O(1) avg	O(log n)

★ Use HashSet for best performance when order doesn't matter.
Use LinkedHashSet when insertion order is important.
Use TreeSet when sorted order is needed.

11. Code Snippet (Print Unique Elements)

```java
// Java
int[] arr = {4, 5, 4, 2, 5, 1, 2, 3};
HashSet<Integer> set = new HashSet<>();
for(int x : arr) set.add(x);
for(int x : set) System.out.print(x + " ");
```

```cpp
// C++
vector<int> v = {4, 5, 4, 2, 5, 1, 2, 3};
unordered_set<int> st(v.begin(), v.end());
for(int x : st) cout << x << " ";
```

12. Quick Tips

- ☑ Set stores only UNIQUE elements.
- ☑ No indexing in Set.
- ☑ Null is allowed once in HashSet.
- ☑ Use Set to eliminate duplicates.
- ☑ Great for membership testing.
- ☑ Very useful in competitive programming!

★ Set = Unique Elements + Fast Operations
Remember: Unordered but Powerful !

Key Takeaway : Use Set when you need uniqueness and fast lookups !



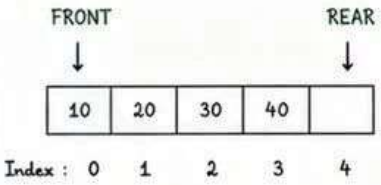
DSA NOTES - 7. QUEUE

1. What is Queue ?

- Queue is a linear data structure that follows FIFO (First In First Out) order.
- Insertion is done at the rear end and deletion is done from the front end.
- It can be implemented using array or linked list.

Think of a queue like a line in a bank. The first person who comes is the first one to be served. :)

2. Representation



Front = 0 (points to 10)
Rear = 3 (points to 40)

3. Basic Operations

- enqueue(x) → Insert element x at the rear.
- dequeue() → Remove and return the front element.
- front() → Get the front element.
- rear() → Get the rear element.
- isEmpty() → Check if the queue is empty.
- isFull() → Check if the queue is full (for array implementation).
- size() → Return number of elements in queue.

4. Implementation

A. Using Array

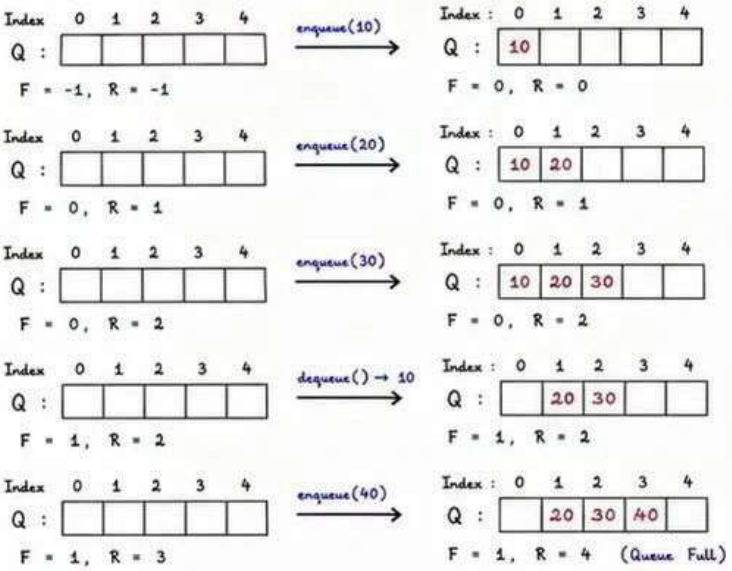
```
# Initialize
queue = [None] * N
front = -1
rear = -1
size = 0
# Enqueue
if rear == N-1:
    print("Queue Overflow")
else:
    if front == -1: front = 0
        rear += 1
        queue[rear] = x
        size += 1
# Dequeue
if front == -1 or front > rear:
    print("Queue Underflow")
else:
    x = queue[front]
    front += 1
    size -= 1
    if front > rear:
        front = rear = -1
    return x
```

B. Using Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, x):
        node = Node(x)
        if self.rear is None:
            self.front = self.rear = node
        else:
            self.rear.next = node
            self.rear = node
    def dequeue(self):
        if self.front is None:
            return None
        temp = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        return temp.data
```

5. Example (Using Array)

Queue size = 5



6. Types of Queue

A. **Simple Queue**
- Standard FIFO queue.

B. **Circular Queue**
- Last position is connected back to the first position.
- Better space utilization.

C. **Priority Queue**
- Elements are dequeued based on priority, not FIFO.

D. **Deque (Double Ended Queue)**
- Insertion and deletion allowed at both ends.

7. Applications of Queue

- ★ CPU Scheduling (Round Robin)
- ★ Printer Spooling
- ★ Breadth First Search (BFS)
- ★ Handling of Requests in Web Servers
- ★ Call Center Systems
- ★ Simulation of Real-life Queues

8. Important Problems

- ★ Implement Queue using Array
- ★ Implement Queue using Linked List
- ★ Circular Queue
- ★ Design Circular Deque
- ★ First Non-Repeating Character in Stream
- ★ Sliding Window Maximum
- ★ Rotten Oranges (BFS)
- ★ Number of Islands (BFS)

9. Complexities

Operation	Array Implementation	Linked List Implementation
enqueue()	O(1)	O(1)
dequeue()	O(1)	O(1)
front()	O(1)	O(1)
isEmpty()	O(1)	O(1)
isFull()	O(1)	O(1)
size()	O(1)	O(1)

(For Circular Queue, all operations are O(1))

10. Queue Using Collections (Python)

```
from collections import deque
q = deque()              # Create empty queue
q.append(10)             # enqueue
q.append(20)
q.popleft()              # dequeue
q[0]                     # front element
q[-1]                    # rear element
len(q)                   # size
len(q) == 0              # isEmpty()
```

11. Quick Tips

- ☑ Queue follows FIFO.
- ☑ Front for deletion, Rear for insertion.
- ☑ Always check for Underflow before dequeue.
- ☑ In array implementation, handle Overflow.
- ☑ Use Circular Queue to utilize space.
- ☑ BFS problems are best solved using Queue.

★ Queue may be simple, but it plays a big role in real life and DSA problems ! Practice more, master better ! :)

Think Smart. Code Better. Optimize Always !

DSA NOTES - 9. MAP / HASH MAP

1. What is Map ?

- Map is a collection of key-value pairs.
- Keys are unique.
- Values can be duplicate.
- Implemented using Hash Table.
- Also known as Dictionary / Associative Array.

Think of Map like a real-life dictionary. You search a word (key) and get its meaning (value) instantly !

2. Characteristics of Map

- ★ Stores key-value pairs.
- ★ Keys are unique, values may repeat.
- ★ No insertion order (unless using LinkedHashMap).
- ★ Allows one null key and multiple null values (in HashMap).
- ★ Average time complexity of basic operations is O(1).
- ★ Uses Hashing for fast lookups.

3. Common Map Operations

Operation	Description
put(key, value)	Insert or update key-value pair.
get(key)	Return value associated with key.
containsKey(key)	Check if key exists.
containsValue(value)	Check if value exists.
remove(key)	Remove key-value pair.
size()	Return number of key-value pairs.
isEmpty()	Check if map is empty.
clear()	Remove all key-value pairs.
keySet()	Return set of all keys.
values()	Return collection of all values.
entrySet()	Return set of all Map entries.

4. Map vs Set vs List

Feature	Map	Set	List / Array
Stores	Key-Value pairs	Unique elements	Elements (ordered)
Duplicates	Keys: No Values: Yes	No	Yes (Allowed)
Order	No order (LinkedHashMap: Yes)	No order (LinkedHashSet: Yes)	Yes
Access	By Key (O(1))	No direct index access	By Index (O(1))
Search	By Key (O(1))	O(1) avg (Hashing)	O(n)
Use Case	When mapping is required	When only unique elements are needed	When order & index matters

5. Implementation in Java (HashMap)

```java
import java.util.*;
HashMap<String, Integer> map = new HashMap<>();

// Insert / Update
map.put("Alice", 25);
map.put("Bob", 30);
map.put("Alice", 26);  // update value
map.put(null, 100);   // null key allowed

// Get Value
int age = map.get("Alice");  // 26

// Check
boolean present = map.containsKey("Bob"); // true
boolean hasValue = map.containsValue(30); // true

// Remove
map.remove("Bob");

// Size
int n = map.size();   // 2

// Iterate
for(Map.Entry<String, Integer> e : map.entrySet()){
   System.out.println(e.getKey() +" -> "+e.getValue());
}
```

6. Implementation in C++ (unordered_map)

```cpp
#include <bits/stdc++.h>
using namespace std;
unordered_map<string, int> mp;

// Insert / Update
mp["Alice"] = 25;
mp["Bob"] = 30;
mp["Alice"] = 26; // update
mp[""] = 100;   // empty string as key

// Get Value
int age = mp["Alice"]; // 26

// Check
bool present = mp.count("Bob");  // 1 if exists else 0

// Remove
mp.erase("Bob");

// Iterate
for(auto &p : mp){
     cout << p.first << " -> "  << p.second << endl;
}
```

7. Example

Input :

N = 5 { (apple, 3), (banana, 2), (apple, 5), (orange, 1), (banana, 4) }

After inserting in Map (key → value):

Key	apple	banana	orange
Value	5	4	1

Duplicate keys overwrite old values !

8. Common Applications of Map

- ★ Frequency counting of elements.
- ★ Storing data in key-value format (like cache).
- ★ Graph adjacency list representation.
- ★ Grouping anagrams.
- ★ Fast lookups in competitive programming.
- ★ Implementing dictionaries, indexes, counters etc.

9. Time Complexity (Average Case)

Operation	HashMap (Java)	unordered_map (C++)
Insertion (put)	O(1)	O(1)
Deletion (remove)	O(1)	O(1)
Search (get)	O(1)	O(1)
containsKey / count	O(1)	O(1)
Iteration	O(n)	O(n)

(Worst Case: O(n) due to Hash Collision)

10. Map in Java - Useful Methods

- getOrDefault(key, defaultValue) → Returns value if key exists, else default value.
- putIfAbsent(key, value) → Inserts only if key not present.
- replace(key, newValue) → Replaces value if key exists.
- replace(key, oldValue, newValue) → Replaces only if old value matches.
- remove(key, value) → Removes only if key maps to given value.

11. Code Snippet - Frequency Count

Java

```java
String s = "aabaccd";
HashMap<Character, Integer> freq = new HashMap<>();
for(char ch : s.toCharArray()){
   freq.put(ch, freq.getOrDefault(ch, 0) + 1);
}
System.out.println(freq); // {a=2, b=1, c=2, d=1}
```

C++

```cpp
string s = "aabaccd";
unordered_map<char, int> freq;
for(char ch : s){
   freq[ch]++;
}
for(auto &p : freq){
    cout << p.first << " -> " << p.second << endl;
}
// a -> 2, b -> 1, c -> 2, d -> 1
```

12. Quick Tips

- ☑ Use Map when you need fast key-based lookups.
- ☑ Always prefer unordered_map / HashMap for average O(1) operations.
- ☑ Use LinkedHashMap (Java) if insertion order is important.
- ☑ Be careful with collisions in custom hash functions.
- ☑ Map is your best friend for counting & grouping problems.

★ Map = Key to Efficient Solutions !

Think Smart. Code Better. Optimize Always !

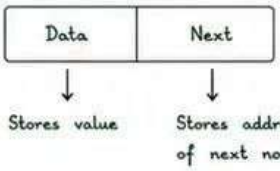
DSA NOTES - 10. LINKED LIST

1. What is Linked List ?

- Linked List is a linear data structure.
- It is a collection of nodes where each node contains data and a reference (link) to the next node.
- Nodes are not stored in contiguous memory locations.
- Dynamic size – can grow or shrink during program execution.

Think of Linked List as a chain where each link (node) points to the next link.

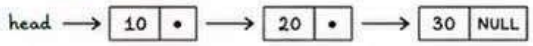
2. Node Structure



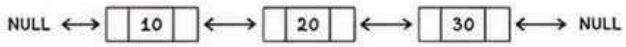

```
// Node in C++
struct Node {
    int data;
    Node* next;
    Node(int x) {
        data = x;
        next = NULL;
    }
};
```

3. Types of Linked List

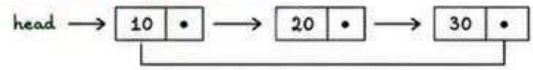
1. Singly Linked List



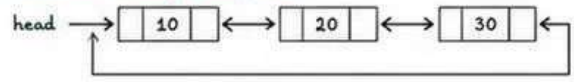
2. Doubly Linked List



3. Circular Singly Linked List



4. Circular Doubly Linked List

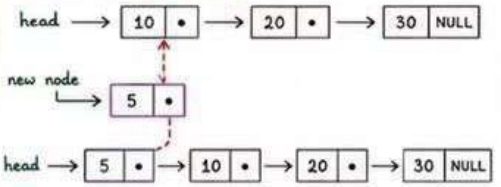


4. Basic Operations (Singly Linked List)

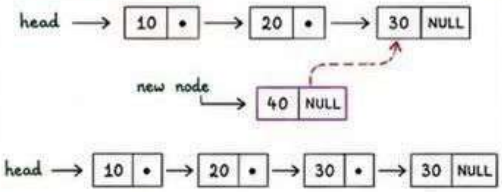
Operation	Description
Traversal	Visit each node and process its data.
Insertion	Add a new node at beginning, end or at a specific position.
Deletion	Remove a node from beginning, end or a specific position.
Search	Find a node with given value.
Length	Count total number of nodes.

5. Insertion in Singly Linked List

(a) Insert at Beginning

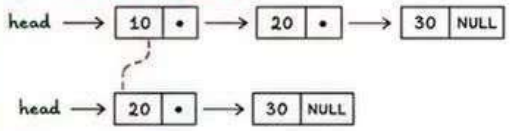


(b) Insert at End

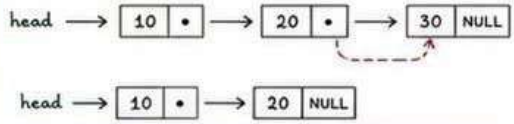


6. Deletion in Singly Linked List

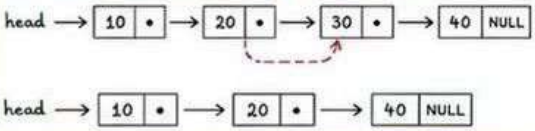
(a) Delete at Beginning



(b) Delete at End



(c) Delete at Position (e.g. position = 2)



7. Traversal

```
void traverse(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
```

Example :

List : 10 → 20 → 30 → NULL

Output : 10 20 30

8. Search

```
bool search(Node* head, int key) {
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data == key)
            return true;
        temp = temp->next;
    }
    return false;
}
```

9. Length of Linked List

```
int length(Node* head) {
    int cnt = 0;
    Node* temp = head;
    while (temp != NULL) {
        cnt++;
        temp = temp->next;
    }
    return cnt;
}
```

★ Linked List is dynamic in size and efficient for insertions and deletions.

10. Common Problems on Linked List

- ★ Reverse a Linked List
- ★ Detect Loop in Linked List (Floyd's Cycle Detection)
- ★ Find Middle of Linked List
- ★ Merge Two Sorted Linked Lists
- ★ Remove Nth Node From End
- ★ Check if Linked List is Palindrome

Detect Loop (Floyd's Algorithm)

```
bool hasCycle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

11. Complexities

Operation	Time Complexity	Space Complexity
Traversal	O(n)	O(1)
Search	O(n)	O(1)
Insertion (given position)	O(n)	O(1)
Deletion (given position)	O(n)	O(1)
Access (by index)	O(n)	O(1)

Where,
n = number of nodes

Master Linked List, Master Interviews !

Practice More. Code Better. Understand Deeply. Crack Any Interview !

DSA NOTES - 11. FAST & SLOW POINTERS

1. What are Fast & Slow Pointers ?

- Fast & Slow Pointers (also called Tortoise and Hare approach) is a technique that uses two pointers moving at different speeds.
- It is mainly used with Linked List to solve problems in O(n) time and O(1) extra space.
- Fast pointer moves 2 steps at a time.
- Slow pointer moves 1 step at a time.

Think of it like a race, fast pointer is the hare and slow pointer is the tortoise ! :)

2. Why is it Useful ?

- ★ Detect cycle in a Linked List.
- ★ Find the middle element of Linked List.
- ★ Find the start of the cycle.
- ★ Solve problems in linear time with constant space.
- ★ Widely asked in coding interviews!

Time Complexity : O(n)
Space Complexity : O(1)

3. How it Works ?

- Initialize two pointers slow and fast at the head.
- Move slow by 1 step.
- Move fast by 2 steps.
- Use the condition based on the problem (meet point, NULL, etc.) to find the result.

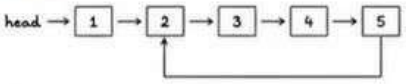
Template (Cycle Detection)

```
slow = head
fast = head
while fast != NULL and fast->next != NULL:
    slow = slow->next          # move 1 step
    fast = fast->next->next    # move 2 steps
    if slow == fast:
        return true   # cycle found
return false          # no cycle
```

4. Detect Cycle in Linked List

Idea : If there is a cycle, fast pointer will eventually meet 1 step pointer.

Example :



Execution :

Step	Slow (1 step)	Fast (2 steps)	
1	2	3	
2	3	5	
3	4	3	
4	5	5	(Meet)

=> Cycle Exists

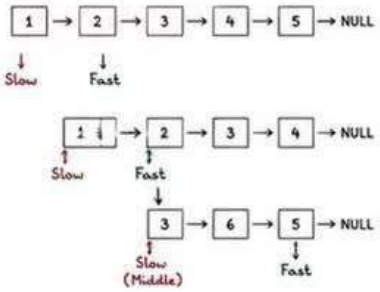
Code (C++)

```
bool hasCycle(Node* head) {
    Node *slow = head;
    Node *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return true;
    }
    return false;
}
```

5. Find Middle of Linked List

Idea : When fast reaches the end, slow will be at the middle.

Example :



Code (C++)

```
Node* findMiddle(Node* head) {
    Node *slow = head;
    Node *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;  // middle node
}
```

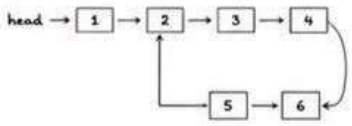
- For even length, this returns the second middle.

6. Find Start of Cycle in Linked List

Idea :

1. First detect cycle and find meeting point.
2. Place one pointer at head, keep the other at meeting point.
3. Move both by 1 step. Where they meet is the start of the cycle.

Example :



Code (C++)

```
Node* detectCycleStart(Node* head) {
    Node *slow = head, *fast = head;
    // Step 1: Detect cycle
    do {
        if (!fast || !fast->next) return NULL;
        slow = slow->next;
        fast = fast->next->next;
    } while (slow != fast);
    // Step 2: Find start
    slow = head;
    while (slow != fast) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}
```

Step 1 : Detect cycle (slow meets fast at node 5)
Step 2 : Move slow to head. Keep fast at node 5.
Step 3 : Move both 1 step at a time.
Meeting point is node 2 (start of cycle).

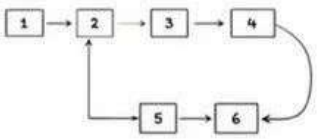
7. Length of Cycle

Idea : After detecting cycle, keep one pointer fixed and move the other until it comes back to the same node, counting steps.

Steps :

1. Detect cycle and get meeting point.
2. Keep one pointer at meeting point.
3. Move the other until it comes back.
4. Count the steps.

Example :



If meeting point is at node 5.
Counting : 5 → 6 → 2 → 3 → 4 → 5
Length = 5

Code (C++)

```
int cycleLength(Node* head) {
    Node *slow = head, *fast = head;
    do {
        slow = slow->next;
        fast = fast->next->next;
    } while (slow != fast);
    int len = 1;
    fast = fast->next;
    while (slow != fast) {
        fast = fast->next;
        len++;
    }
    return len;
}
```

8. Remove Cycle from Linked List

Idea : Find the start of the cycle and the node just before it. Make that node's next = NULL.

Steps :

1. Find start of cycle.
2. Traverse the cycle and find the last node.
3. Break the cycle.

Code (C++)

```
void removeCycle(Node* head) {
    Node *start = detectCycleStart(head);
    if (!start) return;
    Node *temp = start;
    while (temp->next != start)
        temp = temp->next;
    temp->next = NULL;
}
```

9. When to Use ?

- ★ Any problem on Linked List involving cycle or middle.
- ★ To optimize space (O(1) instead of using extra data structures).
- ★ Frequently asked in interviews!

10. Quick Tips

- ☑ Always check fast && fast->next before moving fast pointer.
- ☑ Works only on Linked List.
- ☑ Don't forget edge cases (empty list, single node, two nodes).
- ☑ Master this pattern - it's an interview favorite!

★ Slow & steady wins the race, and finds the solution too ! 👍 :)

Practice More. Code Better. Think Smart. Crack Interviews !

DSA NOTES - 12. SORTING

1. What is Sorting ?

- Sorting is the process of arranging elements in a particular order.
- Common orders :
  Ascending Order (Small → Large)
  Descending Order (Large → Small)
- Sorting is widely used in searching, optimization, indexing, grouping, etc.

Good sorting makes many problems easier & faster !

2. Key Points

★ Sorting helps in efficient searching.

★ Different algorithms have different time and space complexities.

★ In-place sorting → uses O(1) extra space.

★ Stable sorting → keeps relative order of equal elements.

★ Choose algorithm based on constraints (time / space / stability).

3. Comparison Table

Algorithm	Best Time	Average Time	Worst Time	Space	Stable	In-place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes

4. Bubble Sort (Stable)

Idea : Repeatedly swap adjacent elements if they are in wrong order.

Example : Sort [5, 1, 4, 2, 8]

Pass 1 : [1] [4] [2] [5] [8] (8 bubbled up)

Pass 2 : [1] [2] [4] [5] [8] (5 bubbled up)

Pass 3 : [1] [2] [4] [5] [8] (No swap → Sorted)

Code (C++) :

```
for(int i = 0; i < n-1; i++){
    bool swapped = false;
    for(int j = 0; j < n-i-1; j++){
        if(arr[j] > arr[j+1]){
            swap(arr[j], arr[j+1]);
            swapped = true;
        }
    }
    if(!swapped) break; // already sorted
}
```

Best Case : O(n) when already sorted
Worst Case : $O(n^2)$

5. Selection Sort (Not Stable)

Idea : Repeatedly find the minimum element from unsorted part and put it at the beginning.

Example : Sort [64, 25, 12, 22, 11]

Pass 1 : [11] 25 12 22 64

Pass 2 : 11 [12] 25 22 64

Pass 3 : 11 12 [22] 25 64

Pass 4 : 11 12 22 25 64 (Sorted)

Code (C++) :

```
for(int i = 0; i < n-1; i++){
    int minIdx = i;
    for(int j = i+1; j < n; j++)
        if(arr[j] < arr[minIdx])
            minIdx = j;
    swap(arr[i], arr[minIdx]);
}
```

Time Complexity : $O(n^2)$ (All cases)
Space Complexity : O(1)

6. Insertion Sort (Stable)

Idea : Build the sorted array one element at a time by inserting current element at its correct position.

Example : Sort [12, 11, 13, 5, 6]

Pass 1 : [11] [12] 13 5 6

Pass 2 : [11] [12] [13] 5 6

Pass 3 : 5 [11] [12] [13] 6

Pass 4 : 5 6 [11] [12] [13] (Sorted)

Code (C++) :

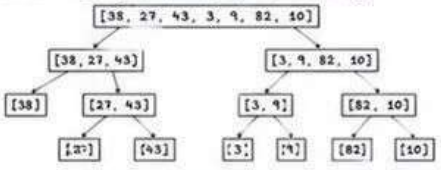
```
for(int i = 1; i < n; i++){
    int key = arr[i];
    int j = i-1;
    while(j >= 0 && arr[j] > key){
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
}
```

Best Case : O(n) (already sorted)
Worst Case : $O(n^2)$

7. Merge Sort (Stable)

Idea : Divide array into halves, sort them and merge the sorted halves.

Divide : [38, 27, 43, 3, 9, 82, 10]



Merge : [3, 9, 10, 27, 38, 43, 82]

Code : (Divide & Conquer)

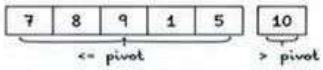
Time Complexity : O(n log n)
Space Complexity : O(n)

8. Quick Sort (Not Stable)

Idea : Choose a pivot, place it at correct position and sort remaining parts.

Example : Sort [10, 7, 8, 9, 1, 5]

Choose pivot = 10



Now sort left part [7, 8, 9, 1, 5]

Choose pivot = 7 → [1, 5] | 7 | [8, 9]

Sort [1, 5] → 1 | 5

Sort [8, 9] → 8 | 9

Final Sorted Array : [1, 5, 7, 8, 9, 10]

Average Case : O(n log n) Worst Case : $O(n^2)$

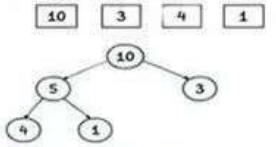
Space Complexity : O(log n) (recursion stack)

9. Heap Sort (Not Stable)

Idea : Build a Max Heap and repeatedly extract the maximum element.

Example : Sort [4, 10, 3, 5, 1]

Step 1 : Build Max Heap



Step 2 : Swap root with last and heapify

Result : [1, 3, 4, 5, 10]

Time Complexity : O(n log n)

Space Complexity : O(1)

10. Counting Sort (Stable)

Idea : Count frequency of each element and place elements in sorted order.

Example : Sort [4, 2, 2, 8, 3, 3, 1]

Step 1 : Count Frequency

Index	0	1	2	3	4	5	6	7	8
Count	0	1	2	2	1	0	0	0	1

Step 2 : Build Output Array

Sorted : [1, 2, 2, 3, 3, 4, 8]

Time Complexity : O(n + k)
(k = range of input)

Space Complexity : O(n + k)

11. When to Use What ?

★ Small or nearly sorted array → Insertion Sort

★ Need stability + guaranteed O(n log n) → Merge Sort

★ In-place + average case fast → Quick Sort

★ Guaranteed O(n log n) + in-place → Heap Sort

★ Small range of integers → Counting Sort

★ Simple but not efficient → Bubble / Selection Sort

There is no one best algorithm.
Choose the right tool for the problem !

12. Quick Tips

☑ Always analyze constraints (n, range, memory).

☑ For integers with small range → Counting Sort is best.

☑ For large data → Quick Sort / Merge Sort preferred.

☑ Stability matters in problems like sorting pairs.

☑ Practice implementations and dry runs.

Practice Sorting.
Build Strong Foundation !

Think Smart. Code Better. Optimize Always !

DSA NOTES - 13. SEARCHING

1. What is Searching ?

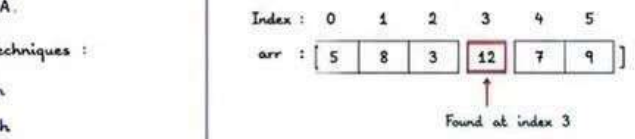
- Searching is the process of finding the position (index) of a target element in a collection.
- It is a fundamental operation and is widely used in DSA.
- Common Search Techniques :
  1. Linear Search
  2. Binary Search

> Searching is simple, but the right choice makes your code fast !

2. Linear Search (Sequential Search)

Idea : Check each element one by one until the target is found or the list ends.

Example : arr = [5, 8, 3, 12, 7, 9] , target = 12



Code (C++) :

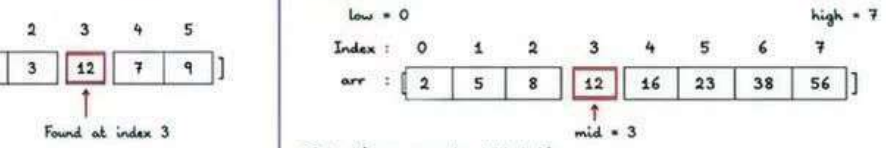
```
int linearSearch(vector<int> &arr, int target){
   for(int i = 0; i < arr.size(); i++){
      if(arr[i] == target) return i;
   }
   return -1;   // not found
}
```

Time Complexity : O(n)
Space Complexity : O(1)

3. Binary Search (Sorted Array)

Idea : Repeatedly divide the search space in half. Compare middle element with target and decide which half to search next.

Example : arr = [2, 5, 8, 12, 16, 23, 38, 56] , target = 16



12 < 16 ⇒ search right half



Code (C++) :

```
int binarySearch(vector<int> &arr, int target){
   int low = 0, high = arr.size()-1;
   while(low <= high){
      int mid = low + (high - low)/2;
      if(arr[mid] == target) return mid;
      else if(arr[mid] < target) low = mid + 1;
      else high = mid - 1;
   }
   return -1;  // not found
}
```

Time Complexity : O(log n)
Space Complexity : O(1)
(For sorted arrays only)

4. Linear Search - Code in Python

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1   # not found
```

5. Binary Search - Code in Python

```
def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = low + (high - low)//2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1   # not found
```

6. Comparison Table

Feature	Linear Search	Binary Search
Requires Sorted Array	No	Yes
Approach	Check one by one	Divide & Conquer
Time Complexity (Best)	O(1)	O(1)
Time Complexity (Avg)	O(n)	O(log n)
Time Complexity (Worst)	O(n)	O(log n)
Space Complexity	O(1)	O(1)
Use Case	Small / Unsorted Data	Large / Sorted Data

7. When to Use What ?

Use Linear Search when :
- Data is unsorted.
- Data size is small.
- Simplicity is preferred.

Use Binary Search when :
- Data is sorted.
- Data size is large.
- Faster search is required.

> ★ Always check if the array is sorted before using Binary Search !

8. Variations of Binary Search

1. First Occurrence (Lower Bound)
   → Find first index of target.
2. Last Occurrence (Upper Bound)
   → Find last index of target.
3. Search Insert Position
   → Find index where target should be inserted to keep array sorted.
4. Count Occurrences
   → Find total number of times target appears in sorted array.

9. Binary Search Variations - Example

arr = [1, 2, 2, 2, 3, 4, 5], target = 2

First Occurrence → index 1
Last Occurrence → index 3
Insert Position (target = 6) → index 7
Count Occurrences → 3

10. Edge Cases

- Empty Array
- Single Element Array
- Target not present
- All elements are same
- Target smaller than all elements
- Target greater than all elements

11. Time Complexity Proof (Binary Search)

Each step reduces the search space by half.

If n = size of array

After 1 step → $\frac{n}{2}$

After 2 steps → $\frac{n}{4}$

After k steps → $\frac{n}{2^k}$

Stop when $\frac{n}{2^k} = 1 \Rightarrow k = \log_2 n$

⇒ Time Complexity = O(log n)

12. Quick Tips

- ☑ Always check if array is sorted.
- ☑ Use int mid = low + (high - low) / 2 to avoid overflow.
- ☑ Binary Search is recursive or iterative.
- ☑ Linear Search works everywhere, Binary Search works on sorted data.
- ☑ Think before using - simplicity or speed ?

13. Practice Problems

- ★ Search in a sorted array.
- ★ Search Insert Position.
- ★ First and Last Position of Element.
- ★ Count Occurrences in Sorted Array.
- ★ Row with minimum 1's (Binary Search on answer).
- ★ Square root of a number (Binary Search).

Practice More. Code Better. Think Smart. Crack Interviews !

DSA NOTES - 14. BINARY SEARCH ON ANSWER

1. What is Binary Search on Answer ?

- It is a technique where we don't binary search the array, instead we binary search on the possible answers.
- We use a condition (predicate) to check if a particular answer is possible or not.
- If it is possible, we try for a better answer.
- If not possible, we try for a smaller answer.

We search the answer space, not the array !

2. When to Use ?

★ When the answer lies in a range.

★ When the answer is monotonic (if we can get X, then we can get any Y >= X).

★ When brute force is difficult but we can verify a solution in O(n) or better.

★ Typical problems :
- Min/Max value problems
- Optimization problems
- Feasibility problems

3. How Does It Work ?

1. Define the answer space [low, high].
2. While (low <= high) :
   - mid = (low + high) / 2
   - Check if mid is a possible answer using a function isPossible(mid)
   - If possible, try to find a better answer. (store ans = mid and move high = mid - 1)
   - Else move low = mid + 1
3. Return the best answer.

Template (C++)

```
long long binarySearchOnAnswer(long long low,
  long long high) {
 long long ans = -1; // or 0, INF depending
 while(low <= high) {
   long long mid = low + (high - low)/ 2;
   if (isPossible(mid)) {
      ans = mid;          // store answer
      high = mid - 1;     // try for better
   } else {
      low = mid + 1;      // try bigger
   }
 }
 return ans;
}
```

4. General Pattern

Step 1 : Identify the answer you are looking for.

Step 2 : Define the search space [low, high].

Step 3 : Define a function isPossible(x) that checks if answer x is possible.

Step 4 : Binary search on the answer range.

Step 5 : Return the best possible answer.

Key Points
- → isPossible(x) should be monotonic.
- → Think: If x is possible, is bigger x also possible?
- → Or if x is not possible, is smaller x also not possible?
- → That property makes binary search valid.

5. Example 1 - Aggressive Cows (Maximum Minimum Distance)

Problem : Given positions of N stalls in a line and C cows. Place cows in stalls such that the minimum distance between any two cows is maximum.

Example :

stalls = [1, 2, 4, 8, 9], C = 3



Choose 1, 4, 8 → min distance = min(3,4) = 3

Try for better answer...

Answer : 3

```
isPossible(dist):
Place 1st cow at stalls[0]
count = 1, last = stalls[0]
for i from 1 to n-1:
  if stalls[i] - last >= dist:
     count++
     last = stalls[i]
return count >= C
```

```
Binary Search On Answer :
low = 0
high = stalls[n-1] - stalls[0]
ans = 0
while(low <= high):
   mid = (low + high) / 2
   if isPossible(mid):
      ans = mid
      low = mid + 1  // try bigger
   else:
      high = mid + 1  // try smaller
return ans
```

6. Example 2 - Koko Eating Bananas (Minimum Eating Speed)

Problem :
Koko has piles of bananas. She can eat max K bananas per hour from a pile. Find the minimum K such that all bananas are eaten within H hours.

Example :

piles = [3, 6, 7, 11], H = 8

Answer : 4

```
isPossible(K):
hours = 0
for pile in piles:
   hours += (pile + K - 1) / K  // ceil division
return hours <= H
```

```
Binary Search On Answer :
low = 1
high = max(bloomDay)          // max possible speed
ans = high
while (low <= high):
      mid = (low + high) / 2
      if isPossible(mid):          // try smaller
         ans = mid
   else
```

7. Example 2 - Minimum Days to Make M Bouquets

Problem :
Given bloomDay[i] and integers m (bouquets) and k (flowers per bouquet), find minimum days to make m bouquets. If not possible, return -1.

Example :

bloomDay = [1, 10, 3, 10, 2], m = 3, k = 1

Answer : 3

```
isPossible(days):
count = 0, flowers = 0
for day in bloomDay:
  if day <= days:
     flowers++
     if flowers == k:
        count++
        flowers = 0
  else:
     flowers = 0
return count >= m
```

```
Binary Search On Answer :
low = min(bloomDay)
high = max(bloomDay)
ans = -1
while(low <= high):
  mid = (low + high)   2
  if isPossible(mid):
     ans = mid
     high = mid - 1  // try earlier day
  else:
     low = mid + 1   // need more days
return ans
```

8. Choosing Boundaries

Problem Type	low	high
Maximum value problem	0 or 1	Large value (1e9, sum, max, etc.)
Minimum value problem	0 or 1	Large value
Answer is in array range	min(arr)	max(arr)
Speed / Rate problems	1	max possible speed
Days / Time problems	min(arr)	max(arr)
Distance problems	0	max(arr) - min(arr)

Set high sufficiently large but within limits to avoid overflow.

9. Common Mistakes

✗ Wrong boundaries.
✗ Wrong condition in isPossible().
✗ Forgetting to handle edge cases.
✗ Infinite loop (wrong mid update).
✗ Using int for large values (use long long).
✗ Returning wrong answer (best vs last).

★ Think carefully about isPossible(mid) - it is the heart of Binary Search on Answer !

10. Quick Tips

☑ Always define what your answer means.
☑ Always write isPossible(x) cleanly and test it separately.
☑ Check monotonicity of your condition.
☑ Binary search for best (min/max) answer.
☑ Dry run small examples before coding.
☑ Use long long for mid and answers.

Binary Search on Answer turns hard problems into simple searches !

Think Smart. Code Better. Optimize Always. Crack Interviews !

DSA NOTES - 15. BIT MANIPULATION

1. What is Bit Manipulation ?

- Bit Manipulation means performing operations directly on bits.
- It helps in optimizing solutions.
- Used in low level programming, cryptography, graphics, networking, competitive programming.

Think in bits,
Solve it like a pro !

2. Number System (Quick Recap)

Decimal	Binary	Representation
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
...	...	...
15	1111	15

1's Complement → Flip all bits (0↔1)
2's Complement → 1's Complement + 1

3. Basic Bitwise Operators

Operator	Symbol	Example (a = 5 (0101), b = 3 (0011))	Result
AND	&	0101 & 0011 = 0001	1
OR	|	0101 | 0011 = 0111	7
XOR	^	0101 ^ 0011 = 0110	6
NOT	~	~0101 ... 1010 (2's comp.)	-6
LEFT SHIFT	<<	0101 << 1 = 1010	10
RIGHT SHIFT	>>	0101 >> 1 = 0010	2

Truth Table

A	B	A & B	A | B	A ^ B
0	0	0	0	0
0	0	0	1	1
1	0	0	1	1
1	1	1	1	0

& → Both bits 1 then 1
| → Any one bit 1 then 1
^ → Different bits then 1
~ → Invert all bits

4. Properties

- ★ a & a = a
- ★ a | a = a
- ★ a & 0 = 0
- ★ a | 0 = a
- ★ a & 1 = a
- ★ a | 1 = 1
- ★ a ^ a = 0
- ★ a ^ 0 = a
- ★ a ^ a ^ b = b
- ★ ~(~a) = a

5. Set, Clear, Toggle a Bit

- Check if ith bit is set (1):
  `(n & (1<<i)) != 0`
- Set ith bit to 1:
  `n | (1<<i)`
- Clear ith bit to 0:
  `n & ~(1<<i)`
- Toggle ith bit:
  `n ^ (1<<i)`

Example : n = 10 (1010), i = 1

Check bit 1 :
10 & (1<<i) = 1010 & 0010 = 0010
!= 0 → Set

Set bit 1 :
10 | (1<<1) = 1010 | 0010 = 1010 (10)

Clear bit 1 :
10 & ~(1<<1) = 1010 & 1101 = 1000 (8)

Toggle bit 1 :
10 ^ (1<<1) = 1010 ^ 0010 = 1000 (8)

6. Power of Two

- A number is power of 2 if only one bit is set.
- Condition :
  `n > 0 and (n & (n - 1)) == 0`

Example :
- n = 1 (0001) → True
- n = 2 (0010) → True
- n = 4 (0100) → True
- n = 6 (0110) → False
- n = 8 (1000) → True

Count set bits == 1

7. Count Set Bits (1's)

Method 1 : Brian Kernighan's Algo
(n & (n-1) removes last set bit)

```
int countSetBits(int n){
    int cnt = 0;
    while(n){
        n = n & (n-1);
        cnt++;
    }
    return cnt;
}
```

Example : n = 13 (1101)
1101 → 1100 → 1000 → 0000 (3 set bits)

8. Clear Last Set Bit

- To clear the rightmost set bit :
  `n & (n - 1)`

Example : n = 20 (10100)
10100 & 10011 = 10000 (16)

Very useful in many problems!

9. Isolate Last Set Bit

- To isolate the rightmost set bit :
  `n & (-n)`

Example : n = 20 (10100)
-n = (~n + 1) = (~10100 + 1)
~10100 = 01011
+1 → 01100
Now n & -n = 10100 & 01100 = 00100 (4)

Gives the value of last set bit

10. Flip All Bits in a Range

- Flip all bits from i to j (0-indexed) :
  Create a mask with 1s in range [i...j] and XOR with n.
  `mask = ((1 << (j - i + 1)) - 1) << i`
  `n = n ^ mask`

Example : n = 23 (10111), i = 1, j = 3
mask = ((1 << 3) - 1) << 1
= (8 - 1) << 1 = 7 << 1 = 1110 (14)
n = 10111 ^ 1110 = 11001 (25)

11. Swap Two Numbers (XOR)

```
a = a ^ b;
b = a ^ b; // now b = original a
a = a ^ b; // now a = original b
```

Example : a = 5 (0101), b = 9 (1001)
a = 0101 ^ 1001 = 1100 (12)
b = 1100 ^ 1001 = 0101 (5)
a = 1100 ^ 0101 = 1001 (9)

(Works only if a and b are variables, not in expressions like arr[i] and arr[j])

12. Check Odd or Even

- If (n & 1) == 1 → Odd
- If (n & 1) == 0 → Even

Example :
n = 7 (0111) → Odd
n = 8 (1000) → Even

13. Multiply / Divide by 2

- Multiply by 2 → Left shift
  `n << 1` (same as n * 2)
- Divide by 2 → Right shift
  `n >> 1` (same as n / 2)

Example : n = 6 (0110)
n<<1 = 1100 (12)
n>>1 = 0011 (3)

14. Negative Numbers in Binary (2's Complement)

- Find 1's complement (flip bits)
- Add 1 to it

Example : -5 (for 4-bit number)
5 = 0101
1's comp = 1010
2's comp = 1010 +1 = 1011 (-5)

Useful for understanding ~ operator

15. Applications

- ★ Number of 1's in binary representation
- ★ Count pairs / subarrays
- ★ Check power of two
- ★ Generate subsets (using bits as masks)
- ★ XOR based problems
- ★ Cryptography & Security
- ★ Low level optimizations.

16. Common Bit Masks

Mask	Binary	Use
0xFFFFFFFF (32-bit All 1s)	1111...1111	All bits set
0x00000000 (32-bit All 0s)	0000...0000	All bits cleared
1 << i	0001 0000..0	Only ith bit set
~(1 << i)	1110 1111..1	All bits 1 except ith bit

17. Quick Tips

- ☑ Always visualize in binary.
- ☑ Use masks smartly.
- ☑ Try to avoid loops, use bit tricks.
- ☑ (n & (n-1)) is your best friend.
- ☑ Practice more problems!

Bit Manipulation may look tricky first, but with practice it becomes super easy and super fast !

18. Practice Problems

- ★ Count set bits in a number
- ★ Check if number is power of 2
- ★ Find missing number in array
- ★ Single Number (Every element twice except one)
- ★ Find two non-repeating numbers
- ★ Flip bits in a given range

Think Smart. Code Better. Optimize Everything. Crack Interviews !

DSA NOTES - 16. BINARY TREE

1. What is Binary Tree ?

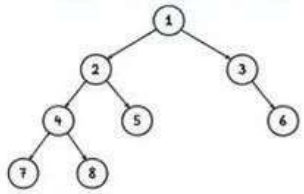
- A Binary Tree is a hierarchical data structure.
- Each node has at most two children : left child and right child.
- The topmost node is called Root.
- Left child < Root < Right child (only in Binary Search Tree).

> A Binary Tree is either empty or consists of a root and two disjoint Binary Trees (left & right subtree).

2. Terminologies

Node : An element of the tree.
Root : The topmost node.
Edge : Connection between two nodes.
Parent : A node that has children.
Child : A node that has a parent.
Leaf Node : A node with no children.
Internal Node : A node with at least one child.
Siblings : Children of the same parent.
Subtree : A node along with its descendants.
Level : Level of root is 0.
Height of Node : Number of edges on longest path from node to leaf.
Height of Tree : Height of root node.

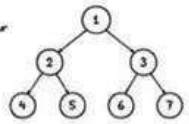
3. Example of Binary Tree



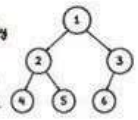
- Root = 1
- Parent of 4 = 2
- Children of 2 = 4, 5
- Sibling of 4 = 5
- Leaf Nodes = 5, 6, 7, 8
- Internal Nodes = 1, 2, 3, 4
- Level of 1 = 0, Level of 4 = 2
- Height of 4 = 1, Height of 2 = 2
- Height of Tree = 3

4. Types of Binary Tree

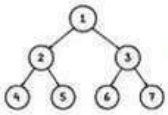
(i) Full Binary Tree
Every node has either 0 or 2 children.



(ii) Complete Binary Tree
All levels are completely filled except possibly the last level which is filled from left to right.

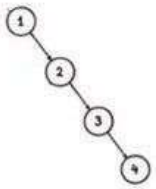


(iii) Perfect Binary Tree
All internal nodes have two children and all leaf nodes are at same level.

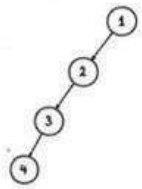


(iv) Skewed Binary Tree
Every node has only one child.

Left Skewed



Right Skewed



5. Properties of Binary Tree

1. Maximum nodes at level $i = 2^i$ $(i >= 0)$
2. Maximum nodes in a tree of height $h = 2^{h+1} - 1$ $(h >= 0)$
3. Minimum nodes in a tree of height $h = h + 1$ $(h >= 0)$
4. For any binary tree, if n_0 = no. of leaf nodes, n_2 = no. of nodes with 2 children, n_1 = no. of nodes with 1 child, Then, $n_0 = n_2 + 1$ and total nodes $= n_0 + n_1 + n_2$
5. For any non-empty binary tree with n nodes, number of edges $= n - 1$

6. Binary Tree Traversals

(i) Preorder (Root, Left, Right)

```
void preorder(Node* root){
    if(root == NULL) return;
    visit(root);
    preorder(root->left);
    preorder(root->right);
}
```

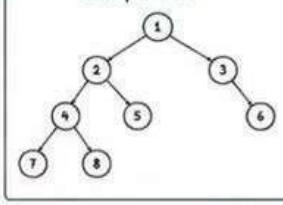
(ii) Inorder (Left, Root, Right)

```
void inorder(Node* root){
    if(root == NULL) return;
    inorder(root->left);
    visit(root);
    inorder(root->right);
}
```

(iii) Postorder (Left, Right, Root)

```
void postorder(Node* root){
    if(root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    visit(root);
}
```

Example Tree



Preorder	: 1 2 4 7 8 5 3 6
Inorder	: 7 4 8 2 5 1 3 6
Postorder	: 7 8 4 5 2 6 3 1
Level Order	: 1 2 3 4 5 6 7 8

7. Level Order Traversal (BFS)

```
void levelOrder(Node* root){
    if(root == NULL) return;
    queue<Node*> q;
    q.push(root);
    while(!q.empty()){
        Node* temp = q.front(); q.pop();
        visit(temp);
        if(temp->left) q.push(temp->left);
        if(temp->right) q.push(temp->right);
    }
}
```

Example (Above Tree): 1 2 3 4 5 6 7 8

8. Height of Binary Tree

```
int height(Node* root){
    if(root == NULL) return -1;
    int lh = height(root->left);
    int rh = height(root->right);
    return 1 + max(lh, rh);
}
```

> Height of empty tree = -1
> Height of tree with one node = 0

9. Count Nodes in Binary Tree

```
int countNodes(Node* root){
    if(root == NULL) return 0;
    return 1 + countNodes(root->left)
             + countNodes(root->right);
}
```

> Time Complexity : O(n)
> Space Complexity : O(h)
> (h = height of tree)

10. Diameter of Binary Tree

```
pair<int, int> diameter(Node* root){
    if(root == NULL) return {0, -1};
    auto L = diameter(root->left);
    auto R = diameter(root->right);
    int dia = max({L.first, R.first,
                   L.second + R.second + 2});
    int ht = 1 + max(L.second, R.second);
    return {dia, ht};
}
```

`diameter(root).first` → diameter

Time Complexity : O(n)

11. Check if Identical Trees

```
bool isIdentical(Node* a, Node* b){
    if(a==NULL && b==NULL) return true;
    if(a==NULL || b==NULL) return false;
    if(a->data != b->data) return false;
    return isIdentical(a->left, b->left) &&
           isIdentical(a->right, b->right);
}
```

12. Check if Mirror Trees

```
bool isMirror(Node* a, Node* b){
    if(a==NULL && b==NULL) return true;
    if(a==NULL || b==NULL) return false;
    if(a->data != b->data) return false;
    return isMirror(a->left, b->right) &&
           isMirror(a->right, b->left);
}
```

13. Binary Tree from Traversals

- **From Inorder & Preorder**
1. First element of preorder is root.
2. Find root in inorder.
3. Build left & right subtree recursively.

- **From Inorder & Postorder**
1. Last element of postorder is root.
2. Find root in inorder.
3. Build left & right subtree recursively.

14. Quick Tips

- ★ Use recursion for most tree problems.
- ★ Understand base case clearly.
- ★ Draw the tree for better visualization.
- ★ Master traversals - they are base for many questions.

> Trees are all about structure. Practice more, visualize more, and code better !

15. Common Applications

- ★ Expression Tree
- ★ Huffman Coding
- ★ File System
- ★ Binary Search Tree
- ★ AI Decision Making
- ★ Network Routing
- ★ Heaps / Priority Queues
- ★ XML / HTML Parsing

Visualize the Tree. Understand the Structure.
Think Smart. Code Better. Crack Interviews !

DSA NOTES - 17. BINARY SEARCH TREE

1. What is BST ?

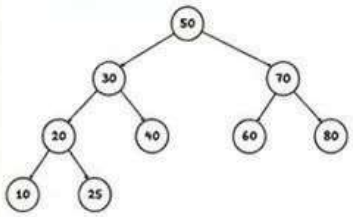
- A Binary Search Tree (BST) is a binary tree in which for every node N :

```
All keys in left subtree of N < N->data
All keys in right subtree of N > N->data
```

- Both left and right subtrees must also be BST.
- Inorder traversal of BST gives sorted order of elements.

BST is the base of many advanced data structures !

2. Example of BST




```
Inorder Traversal   : 10 20 25 30 40 50 60 70 80
Preorder Traversal  : 50 30 20 10 25 40 70 60 80
Postorder Traversal : 10 25 20 40 30 60 80 70 50
```

3. Properties of BST

1. Left Subtree < Root < Right Subtree.
2. Inorder traversal gives sorted sequence.
3. No duplicate keys (by default).
4. Search, Insert, Delete takes O(h) time where h = height of BST.
5. For balanced BST, h = O(log n). For skewed BST, h = O(n).

```
Average Case : O(log n)
Worst Case   : O(n) (skewed tree)
```

4. Search in BST

Idea : Start from root. Compare key with current node and decide the direction.
If key < node->data → go left
If key > node->data → go right
If key == node->data → found

```
Node* search(Node* root, int key){
  while(root != NULL){
    if(key == root->data) return root;
    else if(key < root->data) root = root->left;
    else root = root->right;
  }
  return NULL;  // not found
}
```

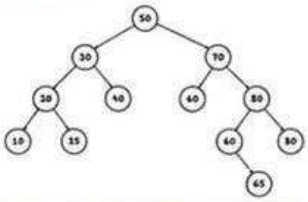
```
Time Complexity : O(h)
Space Complexity : O(1)
```

5. Insert in BST

Idea : Insert like a normal BST. Follow BST property till NULL position and insert the new node.

```
Node* insert(Node* root, int key){
  if(root == NULL) return new Node(key);
  if(key < root->data)
     root->left = insert(root->left, key);
  else if(key > root->data)
     root->right = insert(root->right, key);
  // if equal, do nothing (no duplicates)
  return root;
}
```

Example : Insert 65 in the above BST



6. Delete in BST

Cases :
1. Node is a leaf → delete directly.
2. Node has one child → replace node with its child.
3. Node has two children → replace the node with inorder successor (smallest in right subtree) or inorder predecessor (largest in left subtree) and delete that node.

Algorithm (using inorder successor) :

```
Node* deleteNode(Node* root, int key){
  if(root == NULL) return root;
  if(key < root->data)
     root->left = deleteNode(root->left, key);
  else if(key > root->data)
     root->right = deleteNode(root->right, key);
  else {
     // Node found
     if(root->left == NULL) return root->right;
     if(root->right == NULL) return root->left;
     Node* succ = minValueNode(root->right);
     root->data = succ->data;
     root->right = deleteNode(root->right, succ->data);
  }
  return root;
}
```

```
Node* minValueNode(Node* node){
  while(node->left != NULL)
     node = node->left;
  return node;
}
```

```
Time Complexity : O(h)
Space Complexity : O(h)
  (recursive stack)
Worst Case when
  tree is skewed.
```

7. Traversals in BST

1. Inorder (LNR)
   Left → Root → Right (Sorted Order)
2. Preorder (NLR)
   Root → Left → Right
3. Postorder (LRN)
   Left → Right → Root
4. Level Order (BFS)
   Use Queue

Inorder is most important in BST. It gives sorted elements in O(n) time.

8. Find Min & Max in BST

Minimum (leftmost node)

```
Node* findMin(Node* root){
  if(root == NULL) return NULL;
  while(root->left != NULL)
     root = root->left;
  return root;
}
```

Maximum (rightmost node)

```
Node* findMax(Node* root){
  if(root == NULL) return NULL;
  while(root->right != NULL)
     root = root->right;
  return root;
}
```

```
Time Complexity : O(h)
Space Complexity : O(1)
```

9. Validate BST

Idea : Check if the inorder traversal is strictly increasing.

```
bool isValidBST(Node* root, Node* &prev){
  if(root == NULL) return true;
  if(!isValidBST(root->left, prev)) return false;
  if(prev != NULL && root->data <= prev->data)
     return false;
  prev = root;
  return isValidBST(root->right, prev);
}
```

Call :

```
Node* prev = NULL;
bool ans = isValidBST(root, prev);
```

```
Time Complexity : O(n)
Space Complexity : O(h)
```

10. Height of BST

```
int height(Node* root){
  if(root == NULL) return -1; // height of empty tree = -1
  int lh = height(root->left);
  int rh = height(root->right);
  return 1 + max(lh, rh);
}
```

```
Time Complexity : O(n)
Space Complexity : O(h)
```

11. Inorder Successor & Predecessor

Inorder Successor of a node :
Smallest node in right subtree.
Inorder Predecessor of a node :
Largest node in left subtree.

Successor (iterative) :

```
Node* successor(Node* root, int key){
  Node* succ = NULL;
  while(root != NULL){
    if(key < root->data){
       succ = root;
       root = root->left;
    } else
       root = root->right;
  }
  return succ;
}
```

Predecessor is similar (go left when key > root->data)

12. Construct BST from Sorted Array

Idea : Use middle element as root to keep tree balanced.

```
Node* sortedArrayToBST(vector<int>& arr,
                       int l, int r){
  if(l > r) return NULL;
  int mid = l + (r - l)/2;
  Node* root = new Node(arr[mid]);
  root->left = sortedArrayToBST(arr, l, mid-1);
  root->right = sortedArrayToBST(arr, mid+1, r);
  return root;
}
```

```
Time Complexity : O(n)
Space Complexity : O(log n)
```

13. Balanced BST

If for every node,
|height(left) - height(right)| <= 1
then BST is called Balanced.

Balanced BST ensures O(log n) operations.

Use AVL Tree or Red-Black Tree for self-balancing BST.

Keep your BST balanced for optimal performance !

14. Applications of BST

- ★ Searching efficiently.
- ★ Maintaining sorted data.
- ★ Implementing Sets and Maps.
- ★ Range queries.
- ★ Used in Database indexing.
- ★ Basis for advanced trees like AVL, Red-Black, Splay Trees.

15. Quick Tips

- ☑ Inorder = Sorted Order.
- ☑ Always think : Left < Root < Right.
- ☑ Use recursion for clean code.
- ☑ Handle delete cases carefully.
- ☑ Balance matters for performance.

Master BST, and half of the DSA treas are yours !

Think Smart. Code Better. Build Strong Logic. Crack Interviews !

DSA NOTES - 18. GRAPH

1. What is Graph ?

- A Graph is a non-linear data structure consisting of a set of vertices (nodes) and a set of edges.
- In a graph, edges connect pairs of vertices.
- Graphs are widely used to represent networks, relationships, etc.

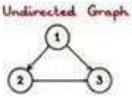
Everything is connected ! That's why graphs are everywhere ☺

2. Terminologies

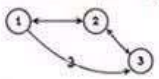
Vertex (Node) : A point in the graph.
Edge : A connection between two vertices.
Adjacent : Two vertices connected by an edge.
Degree : Number of edges incident on a vertex.
Path : A sequence of vertices where each adjacent pair is connected.
Length of path : Number of edges in the path.
Cycle : A path that starts and ends at the same vertex.
Connected : There exists a path between every pair of vertices.

3. Types of Graphs

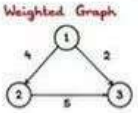
(i) Undirected Graph



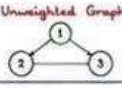
(ii) Directed Graph (Digraph)



(iii) Weighted Graph



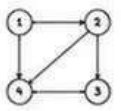
(iv) Unweighted Graph



4. Graph Representation

(i) Adjacency Matrix

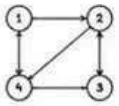
	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	0	1	0	1
4	1	0	1	0



For undirected graph, matrix is symmetric.

(ii) Adjacency List

1 → 2 → 4
2 → 1 → 3
3 → 2 → 4
4 → 1 → 3

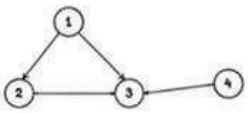


Space : Matrix → $O(V^2)$
List → $O(V + E)$
(V = vertices, E = edges)

5. Degree of a Vertex

- Degree of a vertex = number of edges incident on it.
- In directed graph :
  Indegree = number of edges coming in
  Outdegree = number of edges going out

Example (Undirected Graph)

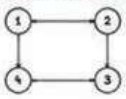


deg(1)=2, deg(2)=2, deg(3)=3, deg(4)=1

6. Path, Cycle & Connected Graph

Path : 1 → 2 → 3 → 4 is a path.
Length of path = 3
Cycle : 1 → 2 → 3 → 1 is a cycle.
Connected Graph : There is a path between every pair of vertices.
Disconnected Graph : At least one pair of vertices is not connected.

Example (Connected Graph)



Example (Disconnected Graph)

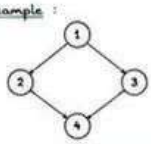


7. Handshaking Lemma

In an undirected graph, sum of degrees of all vertices = 2 × (number of edges)

Proof (Intuition) :
Every edge contributes 1 to the degree of its two endpoints.
So, total contribution = 2E.

Example :



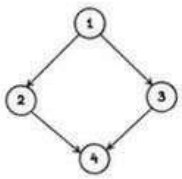
deg(1) = 2
deg(2) = 2
deg(3) = 2
deg(4) = 2

Sum = 8 = 2 × 4 (edges)

8. Directed Graph Concepts

- Indegree of a vertex = number of edges coming into it.
- Outdegree of a vertex = number of edges going out from it.
- Total degree = Indegree + Outdegree.

Example (Directed Graph)



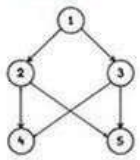
	Indegree	Outdegree
1	0	2
2	1	1
3	1	1
4	2	0

9. BFS (Breadth First Search)

- Visits vertices level by level.
- Uses Queue.
- Time Complexity : $O(V + E)$

Algorithm (BFS from source s)
1. Mark all vertices as unvisited.
2. Push source 's' in queue and mark visited.
3. While queue not empty :
   - Pop front node.
   - Visit it.
   - Push all unvisited adjacent nodes and mark them visited.

Example



BFS from 1 :

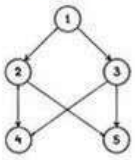
1	2	3	4	5

10. DFS (Depth First Search)

- Goes as deep as possible before backtracking.
- Uses Stack (or Recursion).
- Time Complexity : $O(V + E)$

Algorithm (DFS from source s)
1. Mark all vertices as unvisited.
2. Visit the source 's' and mark visited.
3. For each adjacent unvisited vertex, do DFS recursively.

Example



DFS from 1 :

1	2	4	5	3

11. Difference : BFS vs DFS

BFS	DFS
Uses Queue	Uses Stack / Recursion
Level order traversal	Depth wise traversal
Finds shortest path (in unweighted graph)	May not find shortest path
More memory	Less memory (usually)
Good for shortest path, minimum edges	Good for topological sort, cycle detection, components

12. Cycle Detection

(i) Undirected Graph (Using DFS)
- If we visit a node that is already visited and is not parent of current node, cycle exists.

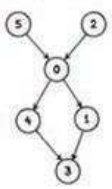
(ii) Directed Graph (Using DFS)
- If we see a back edge (node on recursion stack), cycle exists.

Cycle detection is important in many real-world problems ! ☺

13. Topological Sort (For DAG)

- DAG = Directed Acyclic Graph
- Linear ordering of vertices so that for every edge u → v, u comes before v.
- Uses DFS.

Algorithm :
1. Do DFS for all vertices.
2. Push node to stack when it finishes.
3. Reverse stack → Topological order.



One Topological Order :

5	2	0	4	1	3

14. Applications of Graph

- ★ Social networks (connections)
- ★ Google Maps / GPS Navigation
- ★ Web pages (link structure)
- ★ Network routing
- ★ Dependencies / Scheduling
- ★ AI (state space search)
- ★ Circuit design
- ★ Recommendation systems

Graphs connect the world in real life and in computers ! ☺

15. Common Graph Problems

- ★ Find if path exists between two nodes
- ★ Find shortest path
- ★ Cycle detection
- ★ Topological Sort
- ★ Connected Components
- ★ Minimum Spanning Tree
- ★ Strongly Connected Components
- ★ Graph Coloring

16. Quick Tips

- ☑ Use Adjacency List for sparse graphs.
- ☑ Use Adjacency Matrix for dense graphs.
- ☑ Always mark visited in BFS/DFS.
- ☑ For shortest path in weighted graph, use Dijkstra's Algorithm.
- ☑ Practice more problems. Draw graphs to understand better !



Understand Connections. Master Graphs. Solve Anything.
Think Smart. Code Better. Crack Interviews !



DSA NOTES - 19. DYNAMIC PROGRAMMING

1. What is Dynamic Programming ?

- Dynamic Programming (DP) is an algorithmic technique for solving problems by breaking them down into overlapping subproblems.
- It solves each subproblem once and stores the result to avoid recomputation.
- DP is mainly used for optimization problems.

Think Recursively.
Solve Smartly.
Save Repetition !

2. Key Principles

(i) **Overlapping Subproblems** :
The problem can be broken into subproblems which are solved repeatedly.

(ii) **Optimal Substructure** :
Optimal solution can be constructed from optimal solutions of its subproblems.

(iii) **Memoization** : Top-Down approach using recursion + storing results in a table.

(iv) **Tabulation** : Bottom-Up approach using iteration + storing results in a table.

3. Approaches

(i) **Memoization (Top-Down)**
- Use recursion.
- Store results in dp[]/dp[][]
- Use when state space is large but not all states are needed.

(ii) **Tabulation (Bottom-Up)**
- Use iteration.
- Build table from base cases to final answer.
- Usually more efficient (no recursion stack).

4. General Template

```
// Top-Down (Memoization)
int solve(int i, int j, ...){
    if(base case) return base_answer;
    if(dp[i][j] != -1) return dp[i][j];
    int ans = ...;    // try all choices
    return dp[i][j] = ans;
}

// Bottom-Up (Tabulation)
for(all states in proper order){
    if(base case) dp[state] = base_answer;
    else dp[state] = ...;  // from smaller states
}
return dp[final_state];
```

5. Steps to Solve DP Problems

1. Define the state.
2. Write the recurrence relation.
3. Identify the base case(s).
4. Choose approach (Memoization / Tabulation).
5. Compute the answer.
6. Return the final result.

Key Point
Every DP problem is just a recursion with repetition.

6. Common DP Patterns

Pattern	Meaning	Examples
1D DP	State depends on one parameter	Fibonacci, Climbing Stairs
2D DP	State depends on two parameters	Grid Paths, LCS, Edit Distance
Knapsack DP	Choices with capacity constraint	0/1 Knapsack, Subset Sum
DP on Sequences	Based on prefixes / substrings	LIS, LCS, Palindrome Partition
DP on Intervals	Based on start & end index	Matrix Chain Multiplication
Bitmask DP	State represented using bits	TSP, Assignments
Digit DP	DP on digits of numbers	Count numbers with given property

7. Example 1 - Fibonacci Number

Problem : Find nth Fibonacci number.
Recurrence : F(n) = F(n-1) + F(n-2)
Base Case : F(0) = 0, F(1) = 1

(i) Memoization

```
int fib(int n, int dp[]){
  if(n <= 1) return n;
  if(dp[n] != -1) return dp[n];
  return dp[n] = fib(n-1, dp)
               + fib(n-2, dp);
}
```

(ii) Tabulation

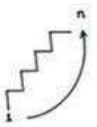
```
int fib(int n){
  if(n <= 1) return n;
  int dp[n+1];
  dp[0] = 0; dp[1] = 1;
  for(int i=2; i<=n; i++)
    dp[i] = dp[i-1] + dp[i-2];
  return dp[n];
}
```

Time Complexity : O(n)
Space Complexity : O(n) → Tabulation
O(n) → Memoization

8. Example 2 - Climbing Stairs

Problem : In how many ways can we climb n steps if we can take 1 or 2 steps at a time ?
Recurrence : ways(n) = ways(n-1) + ways(n-2)
Base Case : ways(0) = 1, ways(1) = 1
Tabulation :

```
int climbStairs(int n){
  if(n <= 1) return 1;
  int dp[n+1];
  dp[0] = 1; dp[1] = 1;
  for(int i=2; i<=n; i++)
    dp[i] = dp[i-1] + dp[i-2];
  return dp[n];
}
```



Time Complexity : O(n)
Space Complexity : O(n) (can be optimized to O(1))

9. Example 3 - 0/1 Knapsack

Problem : Given weights wt[], values val[] and capacity W, find maximum value.
Recurrence :

```
if (wt[i-1] <= W)
  dp[i][W] = max(val[i-1] + dp[i-1][W - wt[i-1]],
                 dp[i-1][W]);
else
  dp[i][W] = dp[i-1][W];
```

Example :
wt = {1, 3, 4, 5}
val = {1, 4, 5, 7}
W = 7

Base Case : dp[0][W] = 0, dp[i][0] = 0
Tabulation :

```
for(int i=0; i<=n; i++){
  for(int w=0; w<=W; w++){
    if(i==0 || w==0) dp[i][w] = 0;
    else if(wt[i-1] <= w)
      dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-]],
                     dp[i-1][w]);
    else
      dp[i][w] = dp[i-1][w];
  }
}
```

Time Complexity : O(nW)
Space Complexity : O(nW)

10. Example 4 - Longest Common Subsequence (LCS)

Problem : Find length of LCS of two strings s1 & s2.
Recurrence :

```
if(s1[i-1] == s2[j-1])
    dp[i][j] = 1 + dp[i-1][j-1];
else
    dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
```

Base Case : dp[i][0] = 0, dp[0][j] = 0
Example : s1 = "abcde", s2 = "ace" → LCS = 3 ("ace")
Time Complexity : O(n*m)
Space Complexity : O(n*m)

i ↓ / j →	""	a	c	e
""	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

DP is mostly about finding the best answer using small answers !

11. Space Optimization

- Many DP problems can be optimized to use less space.
- Convert 2D DP to 1D DP if possible.
- Use rolling arrays.
- Use variables if only previous states are needed.

Example (Fibonacci) :
Instead of dp array, we can use two variables.

```
int fib(int n){
  if(n <= 1) return n;
  int a = 0, b = 1, c;
  for(int i=2; i<=n; i++){
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

Optimize Space,
Optimize Code,
Optimize You !

12. Common DP Problems List

- ★ Fibonacci Number
- ★ Climbing Stairs
- ★ House Robber
- ★ Unique Paths (Grid)
- ★ Minimum Path Sum in Grid
- ★ Triangle Minimum Path Sum
- ★ Coin Change (Min Coins / Ways)
- ★ 0/1 Knapsack
- ★ Subset Sum / Partition Equal Subset Sum
- ★ Longest Increasing Subsequence (LIS)
- ★ Longest Common Subsequence (LCS)
- ★ Edit Distance
- ★ Matrix Chain Multiplication
- ★ Palindrome Partitioning
- ★ Burst Balloons
- ★ Minimum Cost to Cut a Stick

13. When to use DP ?

- ☑ Problem has overlapping subproblems.
- ☑ Recursive solution is recomputing same states.
- ☑ Optimal solution can be built from sub-solutions.
- ☑ Problem constraints allow storing states.

If recursion is TLE → Think DP !
If brute force is slow → Think DP !
DP turns exponential into polynomial !

14. Quick Tips

- ☑ Understand the state clearly.
- ☑ Write down recurrence before coding.
- ☑ Handle base cases carefully.
- ☑ Draw the state diagram / tree.
- ☑ Try small examples before coding.
- ☑ Check for space optimization.

Think Smart. Code Better. Solve Faster. Crack Interviews !